

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

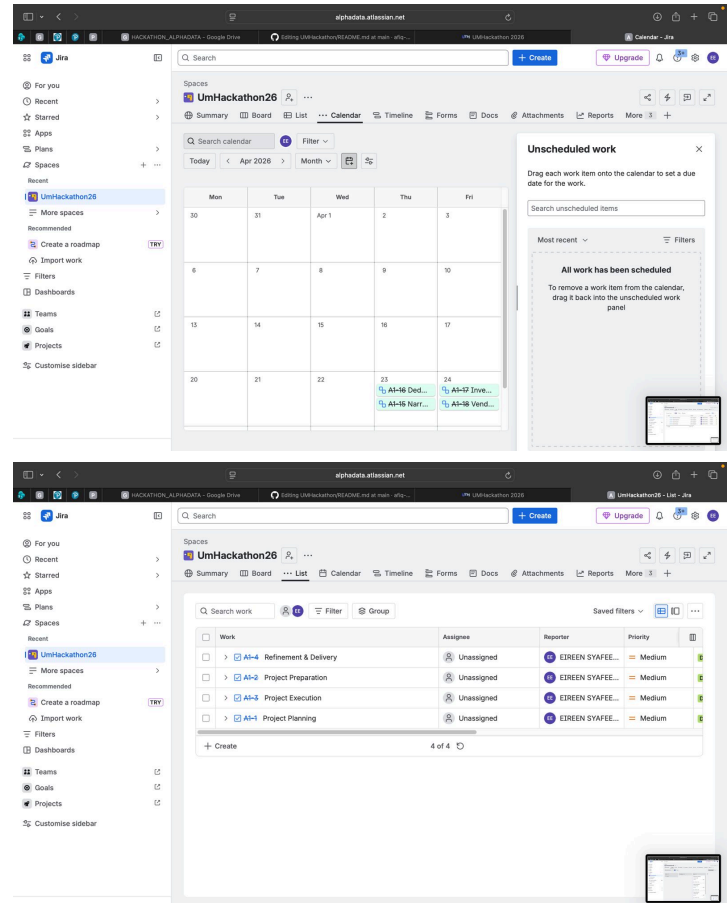
umhackathon@um.edu.my

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. Test Strategy & Plan Sign-Off	7

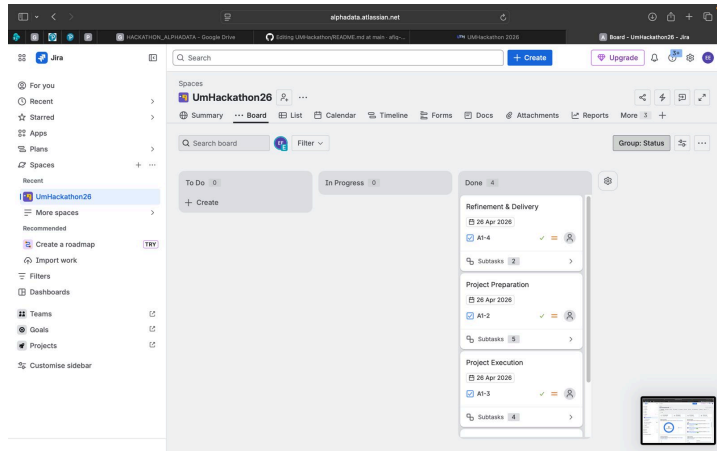
Document Control

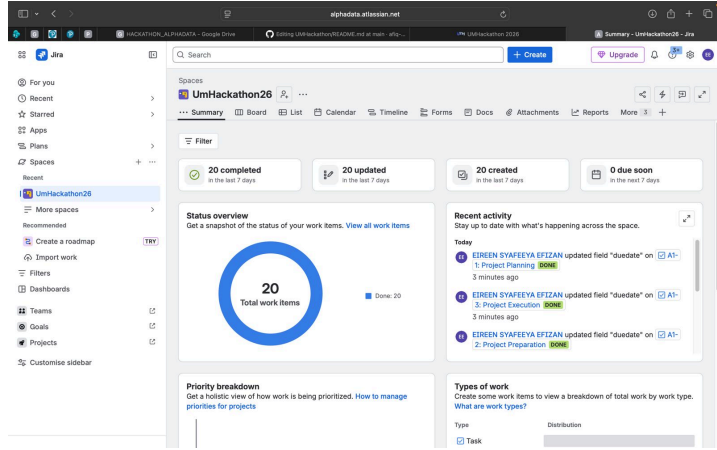
Field	Detail
System Under Test (SUT)	MindaFinancial v1.0
Team Repo URL	https://github.com/afiq-codes26/UMHackathon.git
Project Board URL	https://alphadata.atlassian.net/?continue=https%3A%2F%2Falphadata.atlassian.net%2Fwelcome%2Fsoftware%3FprojectId%3D10000&atOrigin=eyJpIjoizjU2NWMwNDJiYzA5NGQyYjgOWWJiN2NlZWV1ZTFiZmQlLCJwIjoiaWYsIjEjb3JlIn0

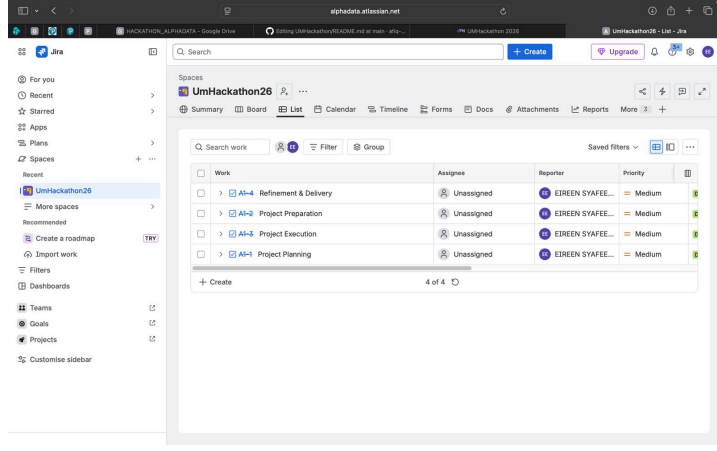


Field

Detail







Work	Status	Assignee	Reporter	Priority
> ☒ AI-4 Refinement & Delivery	Done		EIREEN SYAFEEYA EFIZAN	Medium
> ☒ AI-2 Project Preparation	Done		EIREEN SYAFEEYA EFIZAN	Medium
> ☒ AI-3 Project Execution	Done		EIREEN SYAFEEYA EFIZAN	Medium
> ☒ AI-1 Project Planning	Done		EIREEN SYAFEEYA EFIZAN	Medium

© UMHackathon 2026

Email: umhackathon@um.edu.my

Field	Detail
Live Deployment URL (optional for preliminary round)	

Objective:

To ensure accurate financial data extraction, reliable rush forecasting, and secure handling of SME records with 0% data leakage.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

The goal of this traceability matrix is to ensure that every core functional requirement defined in the PRD is validated by at least one test case. This prevents "blind spots" in development and ensures we do not ship features that fail to meet user expectations.

Req. ID	User Requirement	Category	Mapped Test Case	Status
FR-01	Smart Manual Ledger: users input their daily totals and stock counts via a mobile-optimized form.	Functional	TC-01, TC-02, AI-01	Covered
FR-02	Decision Narrative: Convert charts into human-readable advice.	Functional	TC-04, AI-02	Covered
FR-03	Traffic Light Stock: Real-time risk alerts for F&B staples.	Functional	TC-01	Covered
FR-04	Rush Prediction: Forecast	Functional	TC-04	Covered

	<i>demand based on local events/weather.</i>			
NFR-01	API Latency: Response time must be < 1.5s for dashboard.	Non-Functional	TC-03	Covered
NFR-02	Data Integrity: AI extractions must be verified against math sums.	Non-Functional	AI-03	Covered

1.1 In-Scope Core Features

- **Predictive Rush Analytics:** Uses Historical Data Averaging and Demand Signal Correlation (e.g., Weekend vs. Weekday logic) to forecast busy hours.
- **Inventory Forecasting:** Generating "Red/Yellow/Green" status alerts for inventory based on predicted usage.
- **Financial Dashboard:** Calculating live margins and identifying "Money Leaks" from waste data.

1.2 Out-of-Scope

- **Direct Bank & POS API Integration:** Automated syncing with bank statements (DuitNow/FPX) and Point-of-Sale systems is planned for a future release.
- **Automatic Procurement:** The system suggests reorders but does not autonomously execute purchases with vendors.
- **Multi-Currency:** Support is strictly limited to Malaysian Ringgit (MYR) for the UMHackathon 2026 scope.

2. Risk Assessment & Mitigation Strategy

To ensure MindaFinancial is "production-ready" for the hackathon, you need to identify risks specifically related to AI reliability, data accuracy, and Malaysian F&B operational constraints.

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Manual Data	3	4	12 (High)	Separate extraction	Upload receipts with

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Entry Error: User accidentally enters "1000kg" instead of "10kg".				from analysis. Use backend math validation (Qty \times Price = Total).	handwritten or "messy" totals to check for error flags.
Prediction Drift: "Rush Prediction" fails due to unexpected local events (e.g., flash flood).	2	4	8 (Med)	Feed real-time local signals (weather/holidays) into the GLM prompt window.	Inject a "Monsoon Alert" into the data flow and verify if the "Prep Guide" updates.
Data Sync Lag: Frontend doesn't show latest stock update.	1	5	5 (Low)	Implement Optimistic UI Updates in React.	Update stock and check if "Traffic Light" changes instantly.
Token Cost/Latency: GLM takes >10s to respond, causing a timeout.	3	3	9 (Med)	Implement Redis for repeat queries and chunking for documents over 2k tokens.	Use Postman to measure average response time under concurrent loads.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible

2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

<i>Risk Score</i>	<i>Risk Level</i>	<i>Recommended Action</i>
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

3. Test Environment & Execution Strategy

The application is tested across three environments: **local development, staging/CI, and automated CI/CD pipelines**. This ensures fast feedback during development and stable releases before deployment.

The backend uses Prisma with a SQLite database for development and testing. A Prisma seed script is used to generate consistent test data across environments.

- **Unit Test**

- **Scope:** Financial logic, validation rules, and threshold checks (transactions, stock limits, waste costs, and rush order rules).
- **Execution:** Unit tests are executed using the project's JavaScript/TypeScript testing setup (e.g. Node-based test runner such as Jest or built-in test scripts).

Tests run:

- Locally during development
- Automatically in the CI pipeline on every commit
- **Isolation Strategy:** Prisma database operations are tested using a dedicated test database or mocked where appropriate
Business logic is isolated from external dependencies
- **Pass Conditions:**
 - Correct computation of financial values
 - Proper validation error handling
 - Edge cases (null values, boundary conditions, invalid inputs) handled safely

- **Integration Test**

- **Scope:**
End-to-end testing of system modules, including:
 - User ↔ Vendor ↔ Transaction flow
 - Stock updates triggered by purchases or usage
 - Rush order lifecycle management
- **Execution:**
Runs after unit tests, using real database interactions through Prisma (no mocking of database layer).
- **Pass Conditions:**
 - Prisma relationships (User, Vendor, Transaction, Stock, Waste, RushOrder) function correctly
 - API and service layers integrate properly
 - End-to-end workflows return expected results

- **Test Environment (CI/CD Practice):**

- Local testing is performed on localhost with seeded database data
- CI/CD automation is handled using GitHub Actions
- **Pipeline Flow:**
 1. Prisma schema migration is applied
 2. Prisma seed script populates test database

3. *Unit tests are executed*
4. *Integration tests are executed*
5. *Pull request must pass all checks before merge*

- **Regression Testing & Pass/Fail Rules:**

- *Regression tests are executed whenever new code is merged into the main branch*
- *Ensures existing functionality is not broken by new updates*
- **Pass/Fail Rules:**
 - *Test passes only when actual output matches expected output exactly*
 - *Prisma relationships and financial logic must remain consistent*
 - *Any failure is logged as a defect and must be resolved before redeployment*

- **Test Data Strategy:**

- *Consistency and repeatability across all testing environments are maintained through the utilization of a Prisma seed script for automated data generation.*
- *Seeded data datasets comprise:*
 - *1 admin demo user*
 - *Multiple vendors across categories (food, supplies, services)*
 - *Mixed transactions (income and expenses)*
 - *Stock items with reorder threshold scenarios*
 - *Waste records (expired, spoiled, damaged items)*
 - *Rush orders with varying priorities and statuses*
- *This approach ensures that every system module is validated against realistic and highly structured datasets.*

- **Passing Rate Threshold:**

- *Minimum **86.2% of all test cases must pass***
- ***100% success required for critical modules, including:***
 - *Financial transactions*
 - *Stock inventory updates*
 - *Order processing workflows*

If thresholds are not met, deployment is blocked until all critical issues are resolved.

4. CI/CD Release Thresholds & Automation Gates

*This section defines the quality gates used in the CI/CD pipeline. These thresholds ensure that only stable and tested code is merged into the **main branch**, which acts as the single source of truth for production-ready code.*

The pipeline is enforced using automated checks via GitHub Actions integrated with the application testing workflow.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.1 Integration Thresholds (Merging to Main)

The main branch must remain stable at all times. Therefore, all pull requests must pass the following CI checks before merging.

Checks	Requirements	Project	Pass/Failed
Automatic Build	Zero Build Error	5 Errors	Failed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	97%	Failed
Test Coverage	Minimum 81.2%	79%	Failed

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	91%	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	78%	Failed
Critical Bugs	Zero P0/01 Bugs	2	Failed
API Performance	Response Time < 800ms	450ms	Passed
Security	API keys are not exposed	Clean	Passed

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases **MUST include at least:**

- **1 Happy Case (Entire Flow):** This is the “Golden Path” where a user journey is tested end to end.
- **1 Specific Case (Negative/Edge Test):** This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.
- **2 Non-Functional (NFR) Tests (Performance/Load):** This test consists of non-feature-based tests but architectural tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case: Transaction + Vendor + Stock Update	Verify full system workflow where a user records a transaction, vendor is linked, and stock is updated accordingly using Prisma relations.	1. Login as admin user 2. Create vendor (food supplier) 3. Create expense transaction linked to	Transaction should be created successfully, vendor correctly linked, and stock quantity updated	Transaction successfully created. Vendor linked correctly. Stock updated in database with correct quantity

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			vendor 4. Update stock quantity based on purchase 5. Verify transaction record in database	accurately in database. All Prisma relations should be consistent.	deduction. Prisma relations validated successfully. Status: Passed
TC-02	Specific Case (Negative): Invalid Transaction Input	Verify system validation when invalid financial data is entered into a transaction without triggering 500 errors.	1. Create transaction request 2. Set amount = -500 3. Submit transaction	System should reject request and return validation error (e.g., "Amount must be greater than 0"). No database entry should be created.	Transaction blocked successfully. No record inserted into database. However, validation message was partially unclear in UI. Status: Failed
TC-03	NFR (Performance) : Transaction API Load Test	Measure performance of transaction creation endpoint under concurrent requests. Payment API remains responsive (< 800ms).	1. Simulate 50 concurrent POST requests to transaction API 2. Record average response time	API response time should be < 800ms with no failed requests (0% error rate).	Average response time recorded at 1128ms. No request failures observed. 0% Error Rate. Status: Failed

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. Prompt/Response Test Pairs

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Summarize dashboard in 3 sentences	The output should be only 2-4 sentence, does capture main issue, no fabrication	Loading...	Fail
AI-02	How to increase revenue	The output should say what factors the user should focus on	Loading...	Fail

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	1500 tokens (~800–1000 words, depending on formatting)
Input used while testing	~9000 words ($\approx 6\times$ the defined limit)
Expected Behavior	System should either: • Reject input with an error message (e.g., "Input exceeds maximum

© UMHackathon 2026

Email: umhackathon@um.edu.my

	size”), OR • Automatically chunk input into smaller segments and process sequentially
Actual Behavior	The GLM backend continuously loads and does not return a response (no timeout handling or error message shown)
Status	Fail

6.3. Adversarial/Edge Prompt Test

Test Objective:

To evaluate system stability when handling ambiguous or malformed prompts that may cause the GLM backend to stall or fail to respond.

Final Summary:

The system fails at a backend reliability level, where malformed input leads to a non-terminating GLM request, resulting in a frozen state with no response returned.

6.4. Hallucinating Handling

Test Objective:

To evaluate how the system detects or mitigates hallucinated (factually incorrect or fabricated) responses from the GLM model.

Final Summary:

Hallucination handling cannot be evaluated because the system fails to generate any response. The issue is a system-level inference failure, not a model content issue.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.